DERIVATIVE PRICING

MODULE 7 | LESSON 4

MERTON (1976) MODEL

Reading Time	120 minutes
Prior Knowledge	Volatility surface, Stochastic volatility, Heston model, Merton model: Jump diffusion
Keywords	Stochastic volatility model with stock price jumps

*In Lesson 4 of Module 7, we will implement the **Merton (1976) jump diffusion model** in Python using Monte Carlo methods. Then, we will discuss the comparison with the Black-Scholes framework and the meaning and interpretation of the different parameters.*

As usual, let's start by importing the necessary libraries:

In [1]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as ss
```

1. Implementing the Merton Model

We will start by implementing the Merton model in Python. As you already know, the model has the following SDE:

$$ \begin{equation*} dS_t = \left(r - r_j \right) S_t dt + \sigma S_t dZ_t + J_t S_t dN_t \end{equation*} $$

with the following discretized form:

$$ \begin{equation*} S_t = S_{t-1} \left(e^{\left(r-r_j-\frac{\sigma^2}{2}\right)dt + \sigma \sqrt{dt} z_t^1}+ \left(e^{\mu_j+\delta z_t^2}-1 \right) y_t \right) \end{equation*} $$

where z_t^1 and z_t^2 follow a standard normal and y_t follows a Poisson process. Finally, r_j equals to:

$$ \begin{equation*} r_j = \lambda \left(e^{\mu_j+\frac{\delta^2}{2}}\right)-1 \end{equation*} $$

Remember that, in order to obtain the parameters of the model, we will perform an exercise of calibration to option market prices. This will be, indeed, the first thing we will

cover in the following Stochastic Modeling course. For now, let's assume these parameters as given and equal to:

In [2]:
```
lamb = 0.75  # Lambda of the model
mu = 0.5  # Mu
delta = 0.25  # Delta
```

Let's also assume the following information for the current stock price, number of simulations in our Monte Carlo estimations, etc.

In [3]:
```
r = 0.05  # Risk-free rate
sigma = 0.2  # Volatility
T = 1.0  # Maturity/time period (in years)
S0 = 50  # Current Stock Price

I = 10000  # Number of simulations (paths)
M = 25  # Number of steps
dt = T / M  # Time-step
```

Next, we will calculate the random numbers that we need, together with some variable definitions for later on:

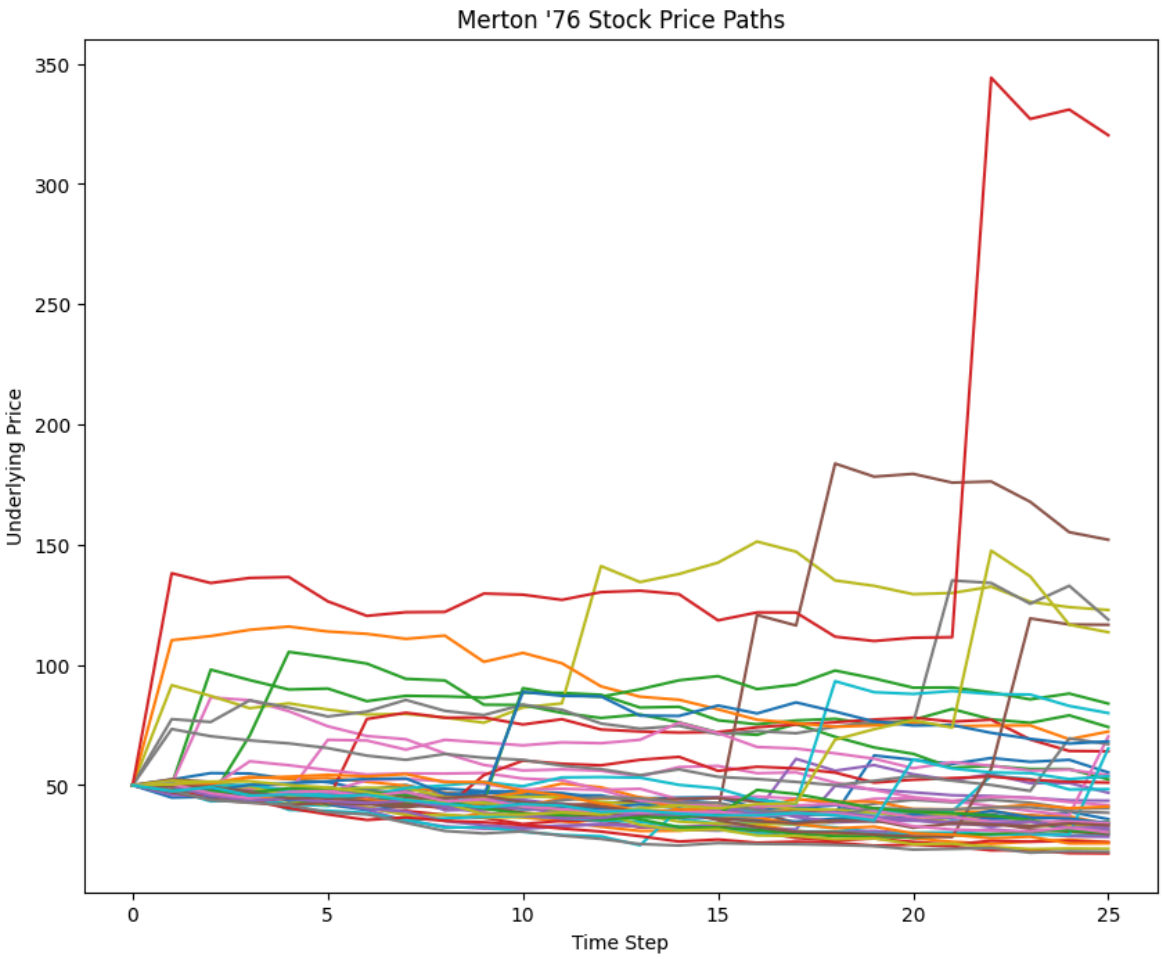
In [4]:
```
SM = np.zeros((M + 1, I))
SM[0] = S0

# rj
rj = lamb * (np.exp(mu + 0.5 * delta**2) - 1)

# Random numbers
z1 = np.random.standard_normal((M + 1, I))
z2 = np.random.standard_normal((M + 1, I))
y = np.random.poisson(lamb * dt, (M + 1, I))
```

With this info, and using the Monte Carlo methods we are familiar with already, we can simulate paths for our stock price under Merton SDE:

In [5]:
```
for t in range(1, M + 1):
    SM[t] = SM[t - 1] * (
        np.exp((r - rj - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z1[t])
        + (np.exp(mu + delta * z2[t]) - 1) * y[t]
    )
    SM[t] = np.maximum(
        SM[t], 0.00001
    )  # To ensure that the price never goes below zero!
```

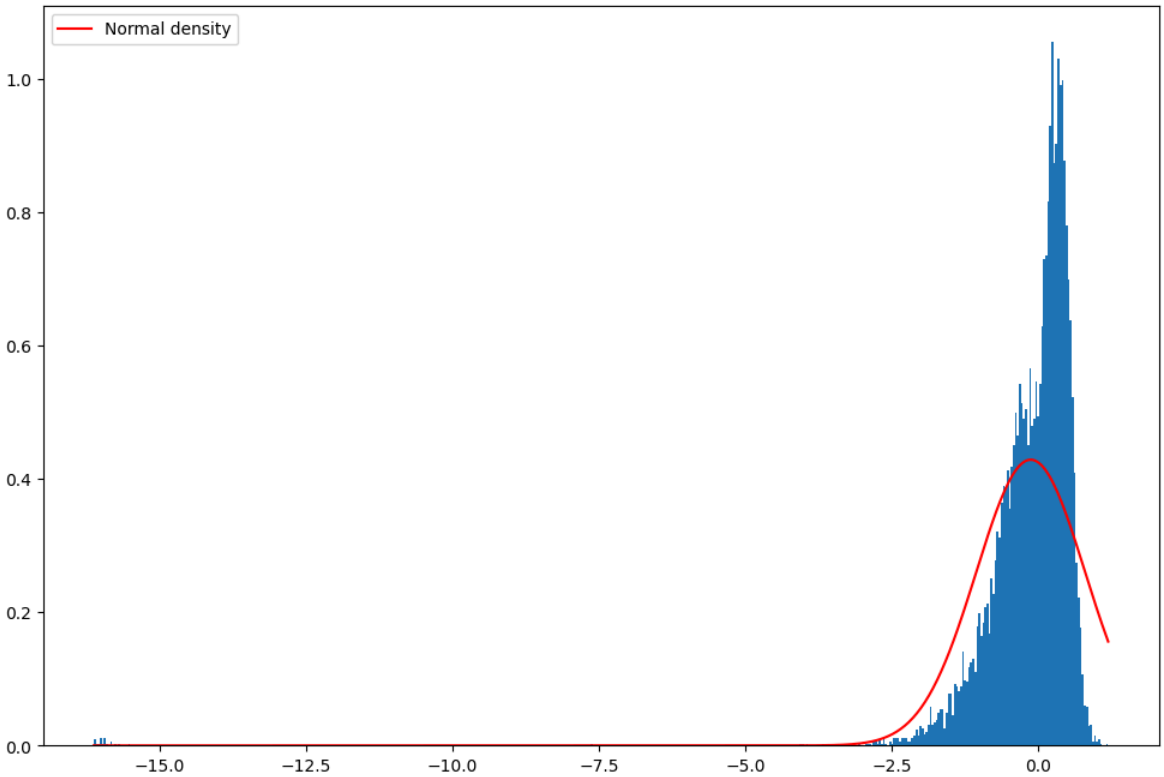
Let's see how stock price evaluation looks for a sample of 50 paths:

In [6]:
```
plt.figure(figsize=(10, 8))
plt.plot(SM[:, 100:150])
plt.title("Merton '76 Stock Price Paths")
plt.xlabel("Time Step")
plt.ylabel("Underlying Price")
plt.show()
```



What is the distribution of stock returns that this model produces?

In [9]:
```
retSM = np.log(SM[-1, :] / S0)
x = np.linspace(retSM.min(), retSM.max(), 500)

plt.figure(figsize=(12, 8))
plt.hist(retSM, density=True, bins=500)
plt.plot(
    x, ss.norm.pdf(x, retSM.mean(), retSM.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```



As you can see, the distribution generated by the Merton model has more kurtosis than a normal distribution and a negative skewness, consistent with the different stylized facts of stock returns. We will, later on, discuss how changing the different parameters modifies this distribution. First, let's compare this result to the returns produced by GBM and Heston.

2. Comparison to Heston and GBM Returns

Next, we will compare the return (and its distribution) generated by other models like Black-Scholes (GBM) or Heston.

Let's start with the former and, when possible, use the same parameters as in the previous example.

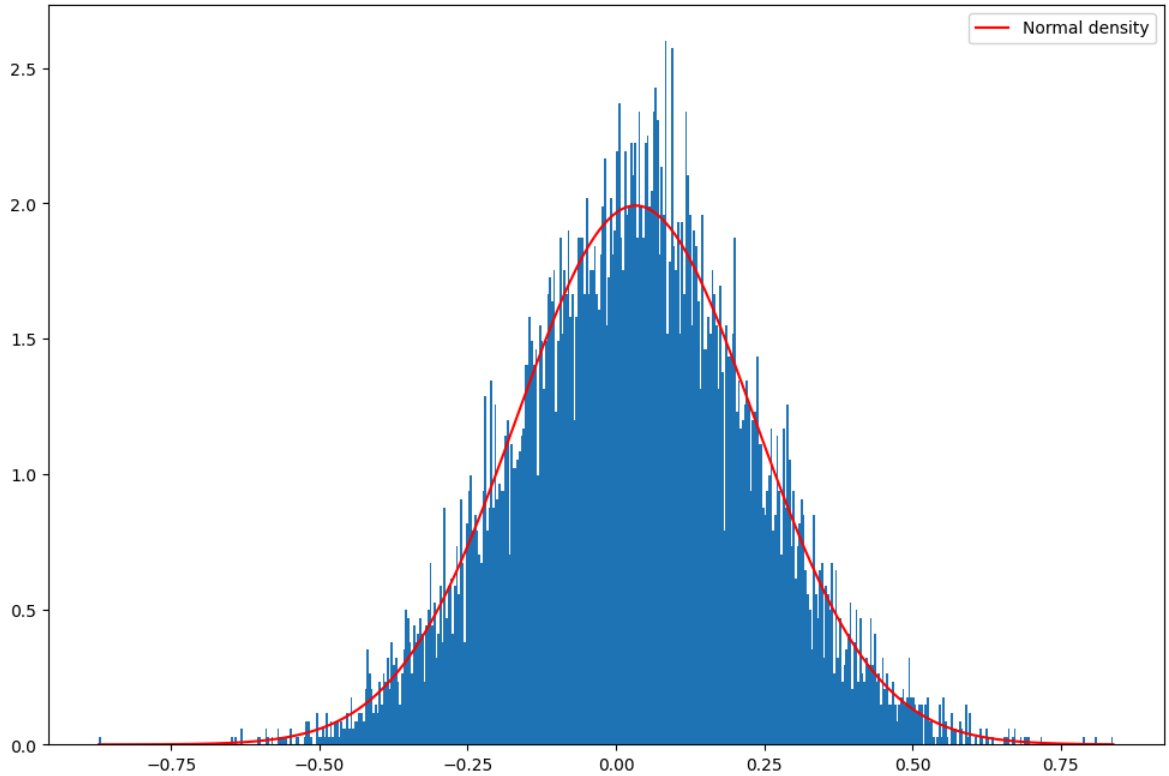
2.1. GBM Returns and Distribution

Let's implement the classic GBM and check the distribution of its returns:

In [10]:
```
S = np.zeros((M + 1, I))
S[0] = S0
for t in range(1, M + 1):
    S[t] = S[t - 1] * np.exp(
        (r - 0.5 * sigma**2) * dt
        + sigma * np.sqrt(dt) * np.random.standard_normal(I)
    )
```

In [11]:
```
retS = np.log(S[-1, :] / S0)
y = np.linspace(retS.min(), retS.max(), 500)
```

In [12]:
```
plt.figure(figsize=(12, 8))
plt.hist(retS, density=True, bins=500)
plt.plot(y, ss.norm.pdf(y, retS.mean(), retS.std()), color="r", label="Normal de
plt.legend()
plt.show()
```



Obviously, what we get here is the classic normal distribution.

2.2. Heston Returns and Their Distribution

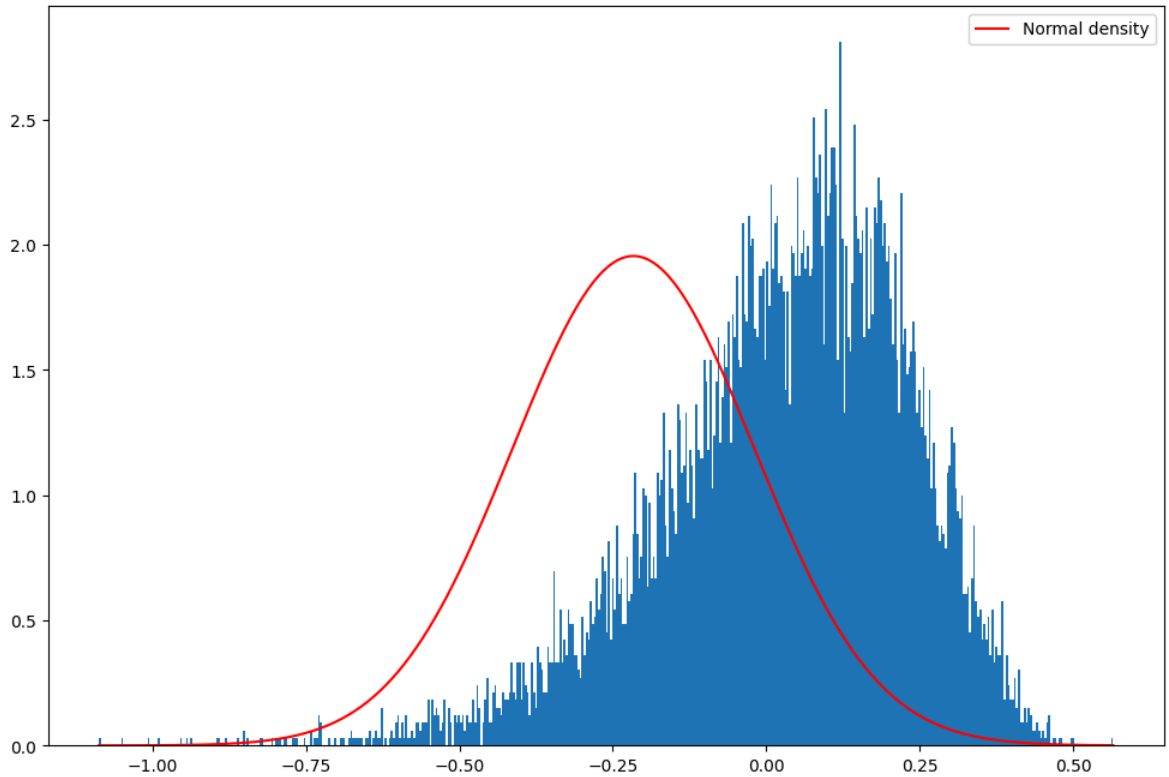
Finally, let's compare this to Heston-produced returns:

In [13]:
```
def SDE_vol(v0, kappa, theta, sigma, T, M, I, rand, row, cho_matrix):
    dt = T / M  # T = maturity, M = number of time steps
    v = np.zeros((M + 1, I), dtype=float)
    v[0] = v0
    sdt = np.sqrt(dt)  # Sqrt of dt
    for t in range(1, M + 1):
        ran = np.dot(cho_matrix, rand[:, t])
        v[t] = np.maximum(0, v[t - 1] + kappa * (theta - v[t - 1]) * dt + np.sqr
    return v

def Heston_paths(S0, r, v, row, cho_matrix):
    S = np.zeros((M+1, I), dtype=float)
    S[0] = S0
    sdt = np.sqrt(dt)
    for t in range(1, M+1, 1):
        ran = np.dot(cho_matrix, rand[:,t])
        S[t] = S[t - 1] * np.exp((r - 0.5 * v[t-1]) * dt + np.sqrt(v[t-1]) * ran

    return S

def random_number_gen(M, I):
```

```
    rand = np.random.standard_normal((2, M+1, I))
    return rand
```

In [14]:
```
# Heston given parameters
v0 = 0.04
kappa_v = 2
sigma_v = 0.2
theta_v = 0.04
rho = -0.9


# Generating random numbers from standard normal
rand = random_number_gen(M, I)


# Covariance Matrix
covariance_matrix = np.zeros((2, 2))
covariance_matrix[0] = [1.0, rho]
covariance_matrix[1] = [rho, 1.0]
cho_matrix = np.linalg.cholesky(covariance_matrix)

# Volatility process paths
V = SDE_vol(v0, kappa_v, theta_v, sigma_v, T, M, I, rand, 1, cho_matrix)

# Underlying price process paths
HS = Heston_paths(S0, r, V, 0, cho_matrix)
```

In [15]:
```
retHS = np.log(HS[-1, :] / S0)
q = np.linspace(retHS.min(), retHS.max(), 500)


plt.figure(figsize=(12, 8))
plt.hist(retHS, density=True, bins=500)
plt.plot(
    q, ss.norm.pdf(y, retHS.mean(), retHS.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```

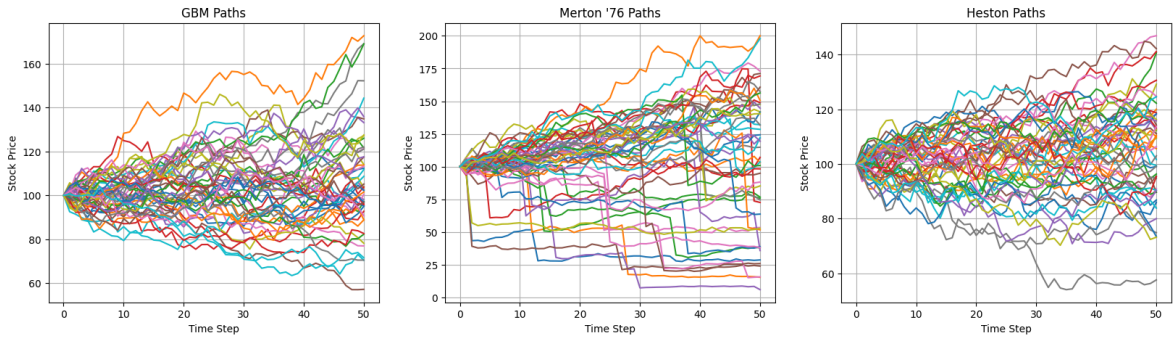


As we have seen in the previous lessons, this is the distribution of returns produced by Heston.

These clear differences between the different models are also evident just by looking at the kind of stock price paths they produce:

In [16]:
```
fig = plt.figure(figsize=(20, 5))
ax1 = fig.add_subplot(131)
ax2 = fig.add_subplot(132)
ax3 = fig.add_subplot(133)

ax1.plot(S[:, :50])
ax1.grid()
ax1.set_title("GBM Paths")
ax1.set_ylabel("Stock Price")
ax1.set_xlabel("Time Step")

ax2.plot(SM[:, :50])
ax2.grid()
ax2.set_title("Merton '76 Paths")
ax2.set_ylabel("Stock Price")
ax2.set_xlabel("Time Step")

ax3.plot(HS[:, :50])
ax3.grid()
ax3.set_title("Heston Paths")
ax3.set_ylabel("Stock Price")
ax3.set_xlabel("Time Step")
```

Out[16]:
```
Text(0.5, 0, 'Time Step')
```



3. Option Pricing under the Different Models

Next, let's explore the consequences of these differences in option pricing. For simplicity, let's assume a European call option with $K=90$ and $T=1$ year. Let's use Monte Carlo methods for this and assume the parameters from before.

3.1. Pricing under Black-Scholes (GBM)

In [17]:
```
def bs_call_mc(S, K, r, sigma, T, t, I):
    data = np.zeros((I, 2))
    z = np.random.normal(0, 1, [1, I])
    ST = S * np.exp((T - t) * (r - 0.5 * sigma**2) + sigma * np.sqrt(T - t) * z)
    data[:, 1] = ST - K

    average = np.sum(np.amax(data, axis=1)) / float(I)

    return np.exp(-r * (T - t)) * average
```

In [18]:
```
print("European Call Price under BS (MC): ", bs_call_mc(S0, 90, r, sigma, T, 0,
```

```
European Call Price under BS (MC):  16.76783971774069
```

3.2 Pricing under Heston

In [19]:
```
def heston_call_mc(S, K, r, T, t):
    payoff = np.maximum(0, S[-1, :] - K)

    average = np.mean(payoff)

    return np.exp(-r * (T - t)) * average
```

In [20]:
```
print("European Call Price under Heston: ", heston_call_mc(HS, 90, r, T, 0))
```

```
European Call Price under Heston:  16.959109347421297
```

3.3 Pricing under Merton

In [21]:
```
def merton_call_mc(S, K, r, T, t):
    payoff = np.maximum(0, S[-1, :] - K)

    average = np.mean(payoff)

    return np.exp(-r * (T - t)) * average
```

In [22]:
```
print("European Call Price under Merton: ", merton_call_mc(SM, 90, r, T, 0))
```

```
European Call Price under Merton:  27.70407381672255
```

Are these results for the price of the options consistent with the distributions we observed?

4. Discussion of Merton Model Parameters

Finally, let's discuss empirically how the different parameters from Merton influence the type and form of the distribution of stock returns. Specifically, we will focus on the parameters λ and μ_j:

- λ → Intensity (frequency) of the jump (shock) to stock prices
- μ_j → Average jump size (can be positive or negative)

As we have already mentioned more than a few times, the actual values that we will use for these parameters will come from a calibration to observed option market prices.

While we will tackle this issue soon, for now, it will be helpful to understand what the sign and magnitude of these parameters mean from an empirical standpoint.

4.1. Changing λ

In the previous example, we used a $\lambda=0.75$, which accounted for the intensity (frequency) of the jumps in interval t. In this case, we expect 0.75 events in a year on average. In other words, the average rate at which jumps occur is 0.75 (75%).

What will happen to the previous distribution of returns if we increase (or decrease) this number?

Let's check it by imposing $\lambda=0.99$:

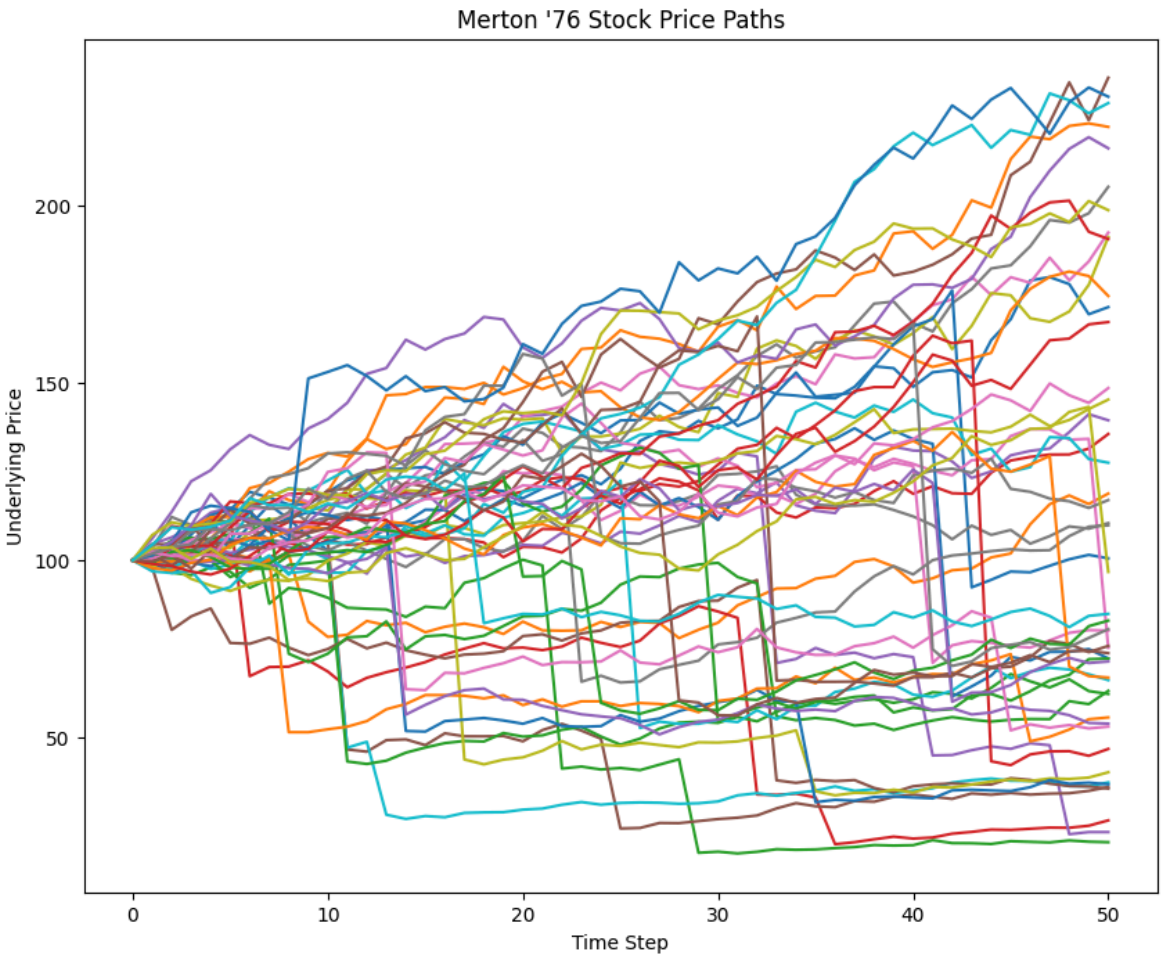
In [23]:
```
lamb = 0.75  # Lambda of the model
mu = 0.6  # Mu
delta = 0.25  # Delta

r = 0.05  # Risk-free rate
sigma = 0.2  # Volatility
T = 1.0  # Maturity/time period (in years)
S0 = 100  # Current Stock Price

I = 10000  # Number of simulations (paths)
M = 50  # Number of steps
dt = T / M  # Time-step

SM = np.zeros((M + 1, I))
SM[0] = S0

# rj
rj = lamb * (np.exp(mu + 0.5 * delta**2) - 1)
```

```
# Random numbers
z1 = np.random.standard_normal((M + 1, I))
z2 = np.random.standard_normal((M + 1, I))
y = np.random.poisson(lamb * dt, (M + 1, I))
```

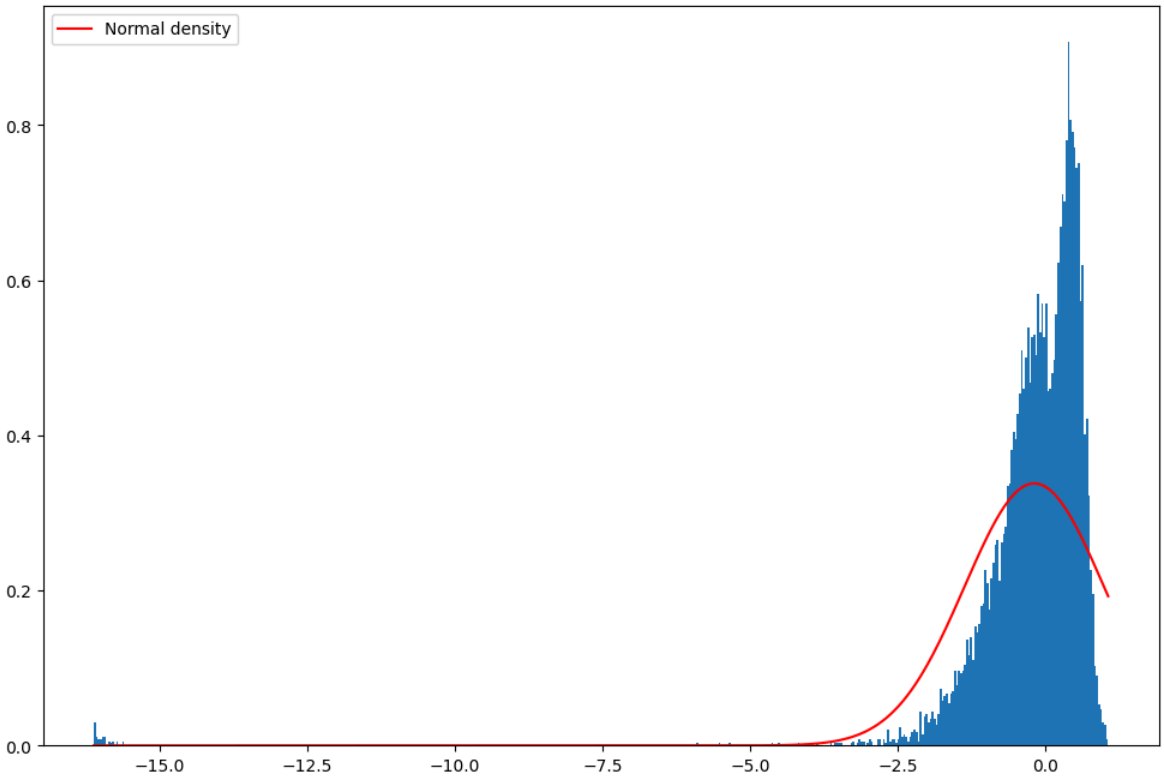
In [24]:
```
lamb = 0.99  # Lambda of the model
mu = -0.6  # Mu
delta = 0.25  # Delta

r = 0.05  # Risk-free rate
sigma = 0.2  # Volatility
T = 1.0  # Maturity/time period (in years)
S0 = 100  # Current Stock Price

I = 10000  # Number of simulations (paths)
M = 50  # Number of steps
dt = T / M  # Time-step

SM = np.zeros((M + 1, I))
SM[0] = S0

# rj
rj = lamb * (np.exp(mu + 0.5 * delta**2) - 1)

# Random numbers
z1 = np.random.standard_normal((M + 1, I))
z2 = np.random.standard_normal((M + 1, I))
y = np.random.poisson(lamb * dt, (M + 1, I))
```

In [25]:
```
for t in range(1, M + 1):
    SM[t] = SM[t - 1] * (
        np.exp((r - rj - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z1[t])
        + (np.exp(mu + delta * z2[t]) - 1) * y[t]
    )
    SM[t] = np.maximum(
        SM[t], 0.00001
    )  # To ensure that the price never goes below zero!
```

In [26]:
```
plt.figure(figsize=(10, 8))
plt.plot(SM[:, 100:150])
plt.title("Merton '76 Stock Price Paths")
plt.xlabel("Time Step")
plt.ylabel("Underlying Price")
plt.show()
```



In [27]:
```
retSM = np.log(SM[-1, :] / S0)
x = np.linspace(retSM.min(), retSM.max(), 500)

plt.figure(figsize=(12, 8))
plt.hist(retSM, density=True, bins=500)
plt.plot(
    x, ss.norm.pdf(x, retSM.mean(), retSM.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```



If you compare this density graph to the one we obtained in the case of $\lambda=0.75$, you will observe that the main difference occurs in the extreme left tail of the distribution. This makes sense, since by increasing λ we are effectively influencing the probability that the stock price experiences more negative returns (jumps).

What if we decrease λ?

4.2. Changing μ_j

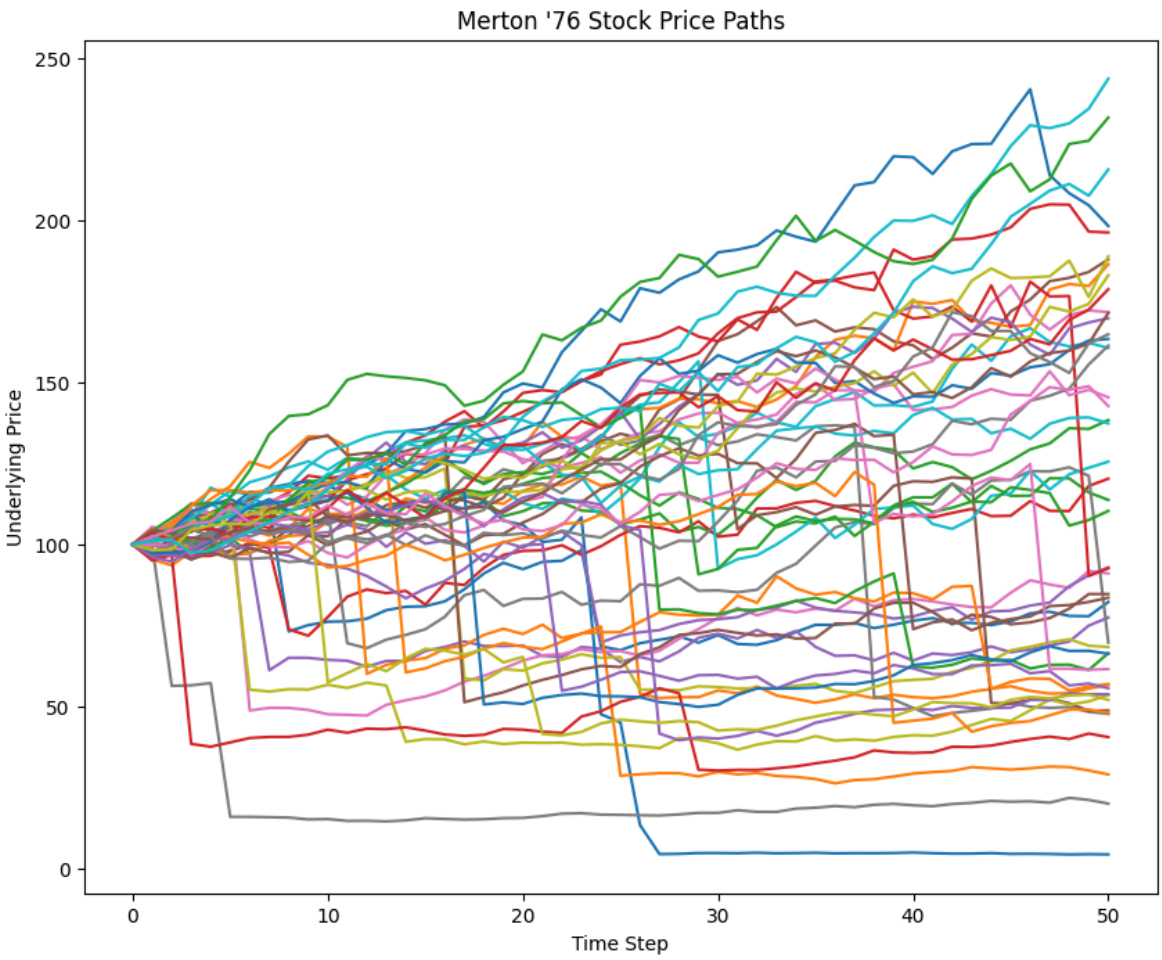
In the previous case, we observe how the distribution's negative skewness increases as a consequence of a higher $λ$. But why the left tail? Essentially, because we have a $\mu_j<0$. That is, the average jump size is negative. This means that every time the stock price experiences a jump, there is a very high likelihood that this is a negative jump.

What if we set $\mu_j>0$?

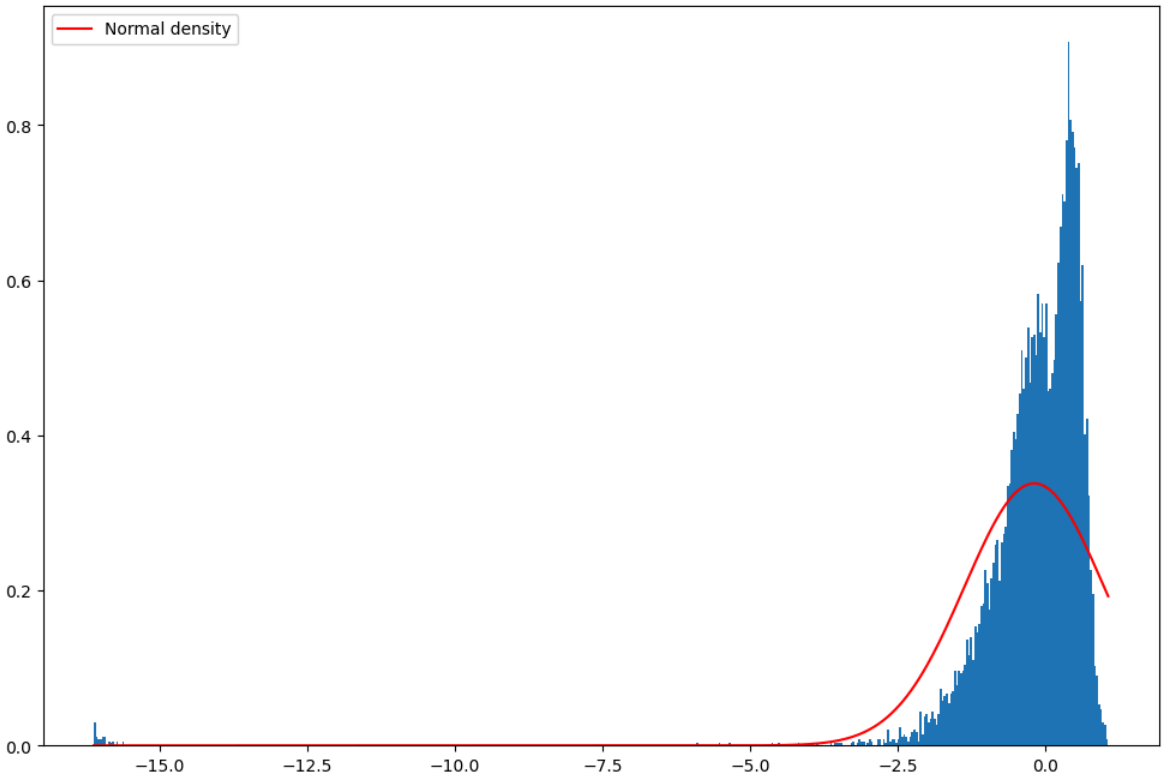
In [28]:
```
for t in range(1, M + 1):
    SM[t] = SM[t - 1] * (
        np.exp((r - rj - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * z1[t])
        + (np.exp(mu + delta * z2[t]) - 1) * y[t]
    )
    SM[t] = np.maximum(
        SM[t], 0.00001
    )  # To ensure that the price never goes below zero!
```

In [29]:
```
plt.figure(figsize=(10, 8))
plt.plot(SM[:, 0:50])
plt.title("Merton '76 Stock Price Paths")
plt.xlabel("Time Step")
```

```
plt.ylabel("Underlying Price")
plt.show()
```



In [30]:
```
retSM = np.log(SM[-1, :] / S0)
x = np.linspace(retSM.min(), retSM.max(), 500)

plt.figure(figsize=(12, 8))
plt.hist(retSM, density=True, bins=500)
plt.plot(
    x, ss.norm.pdf(x, retSM.mean(), retSM.std()), color="r", label="Normal densi
)
plt.legend()
plt.show()
```



Now, as you see, the distribution of returns has skewness (heavier right tail) than normal.

4.3. Changing δ_j

What will happen if we change δ_j? See for yourself!

5. Conclusion

Nicely done! That is all for the Derivative Pricing course. We are not done with these models, however, as we will revisit both of the models covered in this module in the next Stochastic Modeling course. We will not only advance our understanding of model calibration but also look at how we can provide semi-closed form solutions to the pricing problem embedded in Heston and Merton's models!

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.